

Machine Learning in the Unity Game Engine

Omar Elfiki, Alexander Payn, Ashour Kariakous, Long Nguyen, Lorenzo Franca,
Alexandru Mihăilă

20 January 2026

Abstract

This project investigates whether early learning signals and configuration metadata can predict the time-to-threshold for Unity ML-Agents environments. We defined reproducible thresholds using reference runs, then built a diverse dataset of 3,913 runs by varying hyperparameters and algorithms across different hardware. Using this data, we trained a classifier that predicts success with 0.86 to 0.887 accuracy within the first 12% of training. Our results demonstrate that early training behavior is highly predictive of final outcomes, proving to be a reliable method of reducing computational waste by terminating under-performing experiments early and re-purposing resources.

Contents

1	Introduction	2
2	Methodology	2
2.1	Data Collection	2
2.2	Threshold Definition	3
2.3	Data Preprocessing	3
2.4	Machine Learning Prediction	4
2.4.1	Cross Validation	5
2.5	Reproducibility	5
3	Implementation	5
3.1	Unity Environment	5
3.2	Training CLI	6
3.3	Prediction and Modeling CLI	6
3.4	Git & Repository Structure	7
3.4.1	Repository Workflows & Automation	7
4	Experiments	7
4.1	Reference Runs and Threshold Creation	7
4.2	Hyperparameter and Algorithm Diversification	7
4.3	Cross-Hardware Experiments	8
5	Results	8
5.1	Cross-Validation	9
5.2	Prediction	9
5.3	Modeling and Analysis	9
6	Discussion	10
7	Conclusion	11
A	Column Dictionary	13
B	Collected Data Table	14
C	Changes made to increase data diversity	15

1 Introduction

Deep reinforcement learning has become an important framework for training autonomous agents in simulated environments. Tools such as Unity ML-agents provide easy interfaces to experiment with DRL algorithms, resulting in DRL being used widely in robotics, game AI, and control problems.

However, training efficiency and behavior predictability are major challenges. The computational and time cost of training an agent will always be variable. A training run can take minutes or hours depending on factors such as the environment, hyperparameters and steps to train. This unpredictability can complicate experiments, affecting reproducibility, especially on a large-scale implementation

This project attempts to investigate whether it is possible to predict the time-to-threshold of a training run using the initial configuration and early learning-curve signals. Prior investigations have shown that early training dynamics often correlate with final performance, both in supervised learning and deep reinforcement learning. In supervised learning, early loss trajectories have been used to predict convergence and performance (Domhan, T., Springenberg, J. T., & Hutter, F., [July, 2015](#)).

It is worth noting that these correlations have not been studied enough in the context of a multi-environment system like Unity ML-Agents. Additionally, large-scale machine learning research sometimes can result in wasted, failed runs which can be repurposed by a reliable early-prediction system reducing wasted computation and time.

Our formal research question is:

Can we predict the time to reach a reproducible agent performance threshold using initial configuration and early learning-curve signals?

2 Methodology

Our methodology involves collecting data from Unity ML-Agents with supervised machine learning using a Python script to use to predict training outcomes. This is accomplished by defining performance thresholds for each environment. Using these processed datasets, we train classification and regression models to predict threshold reachability and estimate time- and step-to-threshold, validating performance through grouped multi-split cross-validation and analytics.

2.1 Data Collection

We collected a dataset of training runs across several Unity ML-Agents environments using a standardized protocol to ensure data validity and consistency. Each run consists of:

Identification	Run ID, environment, seed
Configuration	DRL algorithm, learning rate, batch size, network architecture, steps
Runtime Metrics	Wall-clock time, CPU/RAM averages
Scalar Logs	Smoothed cumulative reward series, raw episode returns, evaluation returns, episode-level success rate windows

Table 1: Description of data points collected from deep reinforcement learning in Unity ML-Agents

Training Runs

We have executed around 3914 training runs across the duration of the project to date. Each run used:

- A step count as specified by Unity official configurations
- Randomly selected seed
- Official hyperparameter configurations or experimental configuration

Each run produced:

- TensorBoard event logs
- Runtime statistics captured by Python module
- Episodic Reward Information
- Training curves (e.g., Reward Mean, Entropy, Policy Loss, Value Loss)

2.2 Threshold Definition

To predict a *time-to-threshold*, we first needed a definition of what constitutes a successful training run. This required constructing a performance threshold per environment

Reference Runs

For each {environment, algorithm} pair, we conducted 10 long reference runs with the full step count to observe each environment’s full performance

Reference performance R_{env}^*

From the last steps of the reference runs (e.g., last 50k), we computed the **90th percentile** of *episode returns*:

$$R_{\text{env}}^* = 90\text{th percentile of episode returns over the last } N \text{ steps}$$

This value approximates the reproducible asymptotic performance of each environment

Operational threshold T_{env}

We defined the operational threshold as:

$$T_{\text{env}} = \alpha \cdot R_{\text{env}}^*, \quad \alpha = 0.8.$$

This captures the idea of good performance without requiring the run to converge to the absolute maximum recorded.

Detection Criteria

Since learning curves are noisy, we applied smoothing and patience rules:

- Smooth Environment/Cumulative Reward with a moving average window $W \in [5, 10]$.
- The threshold is considered “reached” when the smoothed reward is $\geq T_{\text{env}}$ for:
 k consecutive points (default $k = 3$), *or*
 at least **70%** of a sliding **1k-step** window.

If a threshold is not reached, this is considered a failed run. For failed runs, the best reward and the step is occurred is still recorded to enhance the prediction/classification task.

2.3 Data Preprocessing

To process the data before training any prediction models on it, our workflow involves the following steps:

- First, a script reads through the raw CSV file and removes runs that do not have any entries in threshold related columns.
- Next, the script removes any columns that are not part of the target class for the models.
- The data normalization utilizes a z-score classifier. This classifier indicates how many standard deviations (std) a data point is from the mean of the distribution, which allows uniform prediction on all environments. This also prevents data skew even if anomalous runs are included in the data set.
- Finally, a binary label (0/1) is added to each run to indicate whether or not the threshold was reached during the run.

These steps form the preprocessing workflow, which outputs a more concise, normalized CSV file ready to be used for prediction.

2.4 Machine Learning Prediction

This section explains how we used the data collected from training runs to predict outcomes for the ML-Agents. Our prediction system consists of two supervised-learning tasks:

1. **Classification:** Will a run reach the environment’s performance threshold
2. **Regression:** How long will the run take (steps and seconds) if it succeeds

This two-stage approach avoids producing misleading duration estimates for runs that ultimately fail.

Task 1: Success Classification

The classifier takes as input:

- Environment & Hyperparameters
- Early reward statistics
- Early loss metrics
- Hardware utilization

It outputs:

$$P(\text{success}|\text{early data}) \in [0, 1]$$

We evaluated standard models like logistic regression and random forests and plan to select the best-performing model using validation accuracy and ROC-AUC (C3.ai, [2025](#)).

Task 2: Time-to-threshold Regression

Being conditional on the above task being predicted a success, the regression model estimates:

- Steps to threshold
- Time to threshold (seconds)

Inputs are identical to the classifier, but the outputs are continuous values rather than categorical. Model performance was measured using:

- Mean Absolute Error (MAE)
- Median Absolute Error
- R^2 score

We compared multiple regression models and selected the best-performing one over all metrics (Geeks For Geeks, [13 Jan, 2025](#)).

Feature Selection

To avoid overfitting, we ranked features by importance and correlation with threshold outcomes

We identified the following as the most informative features:

- Early Reward Mean
- Entropy Mean
- Policy Loss Mean
- CPU/RAM Utilization

2.4.1 Cross Validation

To evaluate model robustness and generalization performance, we used a multi-split cross-validation approach. Multi-split cross-validation was selected to reduce dependence on a single random split, provide more stable performance estimates under limited data and ultimately, better reflect real-world deployment variability. The dataset was repeatedly split into training and validation sets using different random partitions, while preserving grouping constraints between related runs. To prevent information leakage, runs originating from the same environment and configuration were grouped together and always assigned to the same split. This ensures that the validation data does not contain runs that are almost duplicates of training data.

For the reachability classifier, we measure **Accuracy** and **F1-score**. These metrics were chosen because threshold reachability is a binary decision problem where both false positives and false negatives are meaningful.

For the regression models, we measure: **Mean Absolute Error (MAE)**, **Root Mean Squared Error (RMSE)**, **R² score**. Together, these metrics show average prediction error, sensitivity to large errors, and give insight to the result.

2.5 Reproducibility

To ensure that our investigation can be reproduced, we maintained use to the same software versions, used a setup script to automatically deploy a virtual environment with consistent dependencies. Additionally, each run was labeled with a unique *RunID* and logged identically each time. All raw logs from training are also stored locally as a backup. Our entire pipeline is documented in our public GitHub repository.

3 Implementation

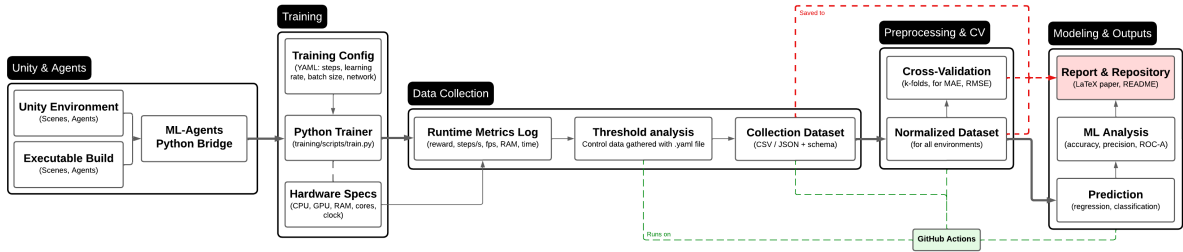


Figure 1: Block diagram of our complete system design

3.1 Unity Environment

We use Unity as the simulation engine for training our agents via ML-Agents. We chose Unity Engine because it provides a realistic and widely used platform for three-dimensional (3D) design with a well-supported bridge to Python for training machine learning agents, enabling straightforward instrumentation of observations and rewards. These properties align with our goal of studying early learning-curve signals across multiple environments. However, Unity can introduce variability in wall-clock time due to render and dependence on hardware and runtime settings. Moreover, learning dynamics may be affected by environment-specific implementation choices. We therefore control simulation settings carefully and interpret time-to-threshold results in light of these constraints. We created a base Unity project containing:

- Example scenes from ML-Agents package
- Additional logging hooks (using StatsRecorder)
- Version-Controlled build settings for reproducibility

Additionally, once scenes and environments were confirmed, we migrated to headless executable builds to run training without the need for the Unity Editor. This allows us to save computational effort and time when collecting training data.

3.2 Training CLI

Environment Setup

To ensure consistency and cross-platform compatibility across team hardware, we implemented an automatic setup script that creates a virtual environment with all packages and dependencies ready to be instantly deployed. The script detects the user’s operating system, then installs the appropriate dependencies directly from a requirements file tailored to the detected platform.

This ensures that training runs use identical dependencies, preventing issues caused by Python version mismatches or local environment conflicts.

Training execution

Training runs are executed using ML-Agents’ *mlagents-learn* trainer (Unity Technologies, 2025), configured using YAML files. Each configuration file contains:

- Algorithm
- Hyperparameters: Learning Rate, Batch Size, Buffer Size
- Network Settings
- Reward Signals

During training, the ML-Agents python package generates:

- TensorBoard Logs
- ONNX Checkpoint files

By default, ML-Agents uses the Proximal Policy Optimization (PPO) algorithm, a widely used reinforcement learning method, though other algorithms were configured during the experiment phase (Unity Technologies, 2021).

The training runner was also extended by adding:

- Timer for wall-clock time per run
- CPU/RAM average utilization logging
- Sequential batch training capability for efficiency
- Auto-commit features to automatically check-in results to repository

Log Parsing and Data Pre-processing

After training is complete, the training CLI extracts our relevant data points and merges them into a structured dataset. The CLI reads event files, normalizes naming conventions, identifies the first step data is available, applies threshold detection and saves the output to the shared dataset CSV file. Snapshots of the CSV are taken and normalized externally before being passed to the prediction module.

3.3 Prediction and Modeling CLI

The prediction module CLI uses normalized datasets derived from data collection to conduct cross validation, model training, and analytics. After the process is called, a directory will be created with the results containing CSV files for predictions, cross-validation and analytics, joblib files for the models, and plots of the confusion matrices and ROC-AUC curves. The process can be run by default or with options using command-line arguments.

3.4 Git & Repository Structure

Our codebase is managed using Git and hosted on GitHub. The repository is organized into the following directories:

unity	Contains the Unity project and environment scenes/scripts
training	Includes the automated venv-setup script, YAML files, and training runner
data	Stores shared datasets, thresholds, summary and validation reports
docs	Contains project documentation (plan, data documentation, report)
models	Contains trained prediction models and ML evaluation scripts

Table 2: Description of unique subdirectories within project repository

The repository comprises multiple markdown files that assist in setting up the project, initiating training runs, and adhering to strict contribution and data logging guidelines to minimize conflicts and maintain the organization of training data.

3.4.1 Repository Workflows & Automation

We use GitHub Actions workflows for multiple aspects of our logging and repository maintenance. When a pull request is opened targeting main, data summarization and validation scripts are automatically deployed to maintain data integrity. Pulls are blocked when validation tests fail. Furthermore, Data preprocessing is completed through workflows to automatically capture and normalize data, then save it to the correct location in the repository. Once preprocessing is complete, the prediction CLI can also be deployed using Actions, producing the results of the run as an artifact to be downloaded by the user. Finally, scheduled workflows create JSON dataset backups and sync the results to the upstream data collection repository.

4 Experiments

This section describes the experiments conducted in an effort to diversify our dataset for prediction. Each experiment was designed to vary environment conditions, hyperparameters and DRL algorithms based on research conducted to capture all behaviors shown by Unity ML-Agents.

Our experiments were split into three main stages:

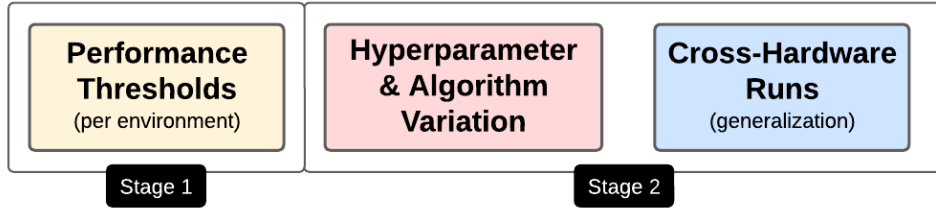


Figure 2: Experiment stages when collecting data for modeling

4.1 Reference Runs and Threshold Creation

First, we conducted 10 long reference runs for each environment. These runs were allowed to run to completion using a high step count to observe full performance. For each environment, we computed the reference reward distribution from the final 50,000 steps and the matching reproducible threshold. All subsequent experiments used these derived thresholds to define the success/failure outcome, steps-to-threshold, and time-to-threshold.

4.2 Hyperparameter and Algorithm Diversification

Once we established our thresholds, the next goal was to add data to our dataset that has wide variability in training dynamics. This is essential to generalize prediction across different conditions. To achieve this,

we modified key hyperparameters in the configuration files. Each experiment involved modifying one of the following:

Learning Rate

We conducted experiments for selected environments with the learning rate halved and doubled relative to the official learning rate. Learning rate strongly impacts stability and efficiency. Variation in learning rate was designed to help us capture both fast-learning and unstable runs.

Batch and Buffer Size

We conducted experiments with the batch and buffer size increased to examine gradient noise, reward smoothness and convergence time. The adjusted buffer sizes were always proportional to the batch size.

Hidden Layers

Some experiments involved a change in the number of hidden units (neurons). Network capacity can impact the time needed for a policy to reach useful performance and hence the total time to convergence.

Entropy

We tested both low and high entropy configurations to examine convergence speed and exploration depth. Entropy strongly influences early training signals and therefore predictive behavior.

Algorithm

While PPO was used as the main DRL algorithm, we also evaluated runs using the SAC (Soft Actor-Critic) algorithm where it was applicable. Including distinct algorithmic behavior improves the robustness of classification and regression models.

4.3 Cross-Hardware Experiments

To evaluate the generalization of the prediction system, we performed additional runs using the original, unmodified configurations on shuffled hardware compared to the first stage of experimentation. This is to account for hardware differences such as: CPU model and speed, memory size and speed, operating system and background process interference. A model trained on single machine samples can risk overfitting to hardware artifacts. A full list of changes can be found under Appendix C.

5 Results

A dictionary explaining each column along with an excerpt of the collected data can be found under appendix A and B, respectively. Full results are available under the `data/` directory of the repository. Our official collected_results.csv has 3913 runs. Normalized snapshots can be found in the `data/normalized` directory

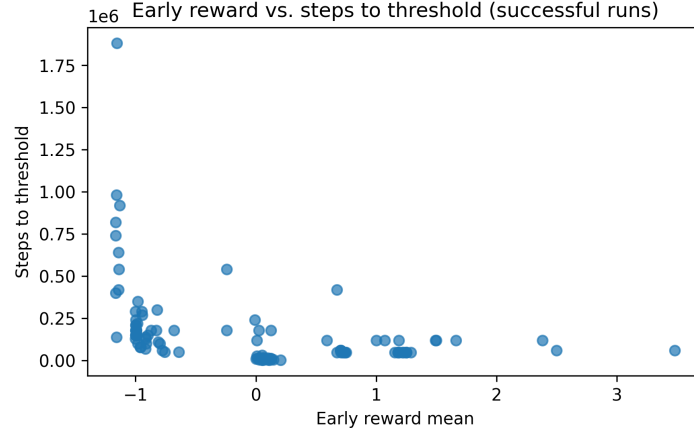


Figure 3: Early reward mean versus steps to threshold across training runs.

5.1 Cross-Validation

The Cross-Validation table presented shows the results of a 5-fold cross-validation across the entire normalized dataset for the two regression tasks:

fold	accuracy	f1	steps mse	steps mae	time mse	time mae
0	0.883117	0.932635	8665995544.09	20792	16502.92	29.22089
1	0.883117	0.933135	8010460316.88	30778.43	72074.09	39.27999
2	0.886866	0.935603	1289652796.43	18768.29	2217.341	25.69073
3	0.86736	0.923538	634909461.04	16378.03	1311.692	22.96514
4	0.859558	0.91904	1561773206.31	19857.36	824.454	21.39607

Table 3: Cross-Validation results per fold; collected on the normalized dataset

5.2 Prediction

Inputs (X)	environment	3DBall(A)	3DBall(B)	Basic(A)	Basic(B)	WallJump(A)	WallJump(B)	PushBlock(A)	PushBlock(B)
	steps	100000	500000	50000	50000	2000000	1000000	20000000	2000000
	learning rate	8.94E-05	0.000128	0.00015	0.0006	0.0003	0.0003	0.0003	0.0003
	batch size	116	166	32	32	128	128	128	128
	buffer size	21855	31142	256	256	2048	2048	2048	2048
	early reward mean	-0.49222	-0.2228	0.881728	-1.64867	-0.13726	0.215781	0.797179	-1.2357
	entropy mean	0.723511	0.670411	1.039967	-1.07162	-1.90542	1.345925	-0.32919	-0.59936
Targets (Y)	steps to threshold	96000	252000	14000	NaN	140000	NaN	120000	180000
	time to threshold	104.9	108.9	73.2	NaN	159.3	NaN	71.5	157.4
	run reached threshold	1	1	1	0	1	0	1	1
Prediction	proportion threshold reached	0.007038	0.780944	0.937602	0.258198	0.877379	0.067434	0.146332	0.901188
	predicted steps to threshold	103774.5	124874.8	11229.89	4874.665	1106126	1867168	47695.28	457491.2
	predicted times to threshold	123.9866	46.79189	54.83886	19.45581	1473.397	771.8213	22.03614	634.3982

Table 4: Prediction results of logistic (classification) and linear (regression) models

5.3 Modeling and Analysis

	3DBall	Basic	WallJump	PushBlock
Accuracy	0.917112	1	1	0.75
Precision	0.998423	1	1	1
ROC	0.946793	1	1	N/A ¹

Table 5: Analysis results per environment

¹ROC-AUC is undefined in the case of one-class entropy

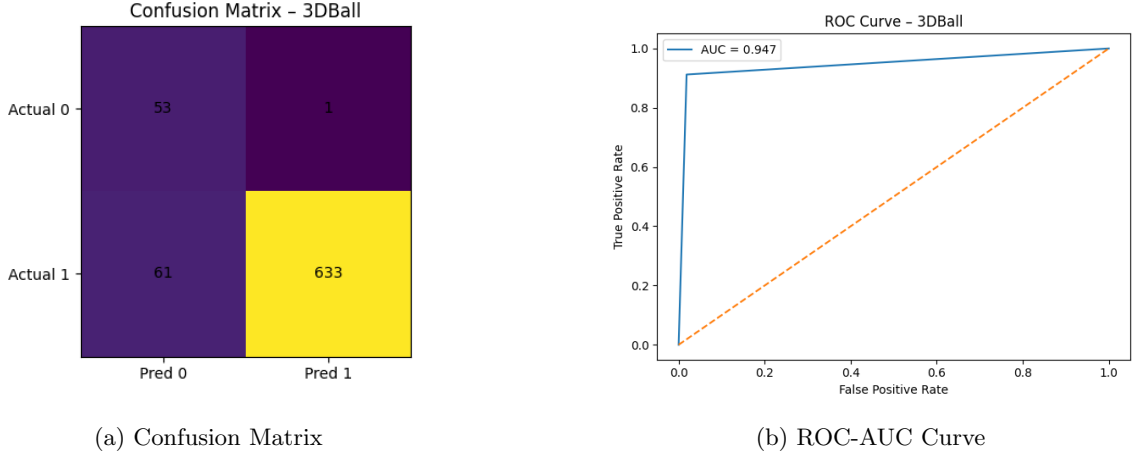


Figure 4: Analysis visualization - filtered on 3DBall

6 Discussion

The cross-validation results in Table 3 show that the classification model has relatively good stability because the accuracy and F1 only fluctuate within a narrow range between folds. Conversely, the regression metrics (MAE for steps/time) fluctuate more widely, suggesting that the prediction of “how long/how many steps” is sensitive to the data split and sample distribution within each fold. Because cross-validation was performed on a multi-environment dataset with an uneven sample distribution across environments, the CV figures reflect overall performance; therefore, further results for each environment are needed to understand where the model performs well/poorly.

The per-environment table (Accuracy, Precision, ROC-AUC) presented in Table 5.3 shows that classification performance varies significantly between environments. On 3DBall (the environment with the largest sample size), the model achieved an accuracy of approximately 0.92 with an ROC-AUC of approximately 0.95 (Figure 4), indicating good and relatively reliable “reach vs. not reach” discrimination due to sufficient data for learning and evaluation. In contrast, some environments such as Basic and WallJump yielded near-perfect scores (accuracy/precision/ROC-AUC = 1.0). However, because data collection was primarily focused on 3DBall, the “perfect” results in other environments likely stem from the very small sample sizes, making the metric less stable and prone to false positives, as just a few correct predictions are enough to achieve a score of 1.0 (Table 5.3).

In review, classification performance ranged from 0.860–0.887 for accuracy and 0.919–0.936 for F1 between folds (Table 3). For regression, steps MAE ranged from ~18.8k–30.8k steps, and time MAE ranged from ~21.4–39.3 seconds, indicating that prediction error varied across folds (Table 3).

The gap between classification and regression performance reflects the difficulty of tasks. Deciding whether a run will eventually reach a threshold is a binary problem that is relatively robust to noise in early learning signals, whereas predicting the time-to-threshold requires modeling a more intricate system influenced by hyperparameters and hardware, as well as the environment’s design. This difference in complexity explains the higher variance observed in regression MAE across folds (Table 3).

Additionally, several limitations constrain the generalizability of these findings. Our dataset is heavily skewed towards 3DBall, providing robust statistics for that environment but risking bias toward this specific environment’s dynamics (Table 5.3). Wall-clock time is inherently hardware dependent, and despite cross-hardware experiments, time predictions remain more sensitive to machine variability than step-based metrics. However, despite these constraints, the results show a practically useful pattern that early-stage classification is stable enough to support early stopping, and regression still offers time-to-threshold estimates that help plan the allocation and usage of resources (Figure 4).

7 Conclusion

Our research successfully demonstrates the main predictors of training outcomes within the Unity ML-Agents framework, those being the learning-curve dynamics and configuration metadata. As such, we have created a standardized metric for success by establishing operational thresholds based on reference performance, allowing us to evaluate the predictive modeling.

Our findings indicate that success classification is stable across diverse training conditions, achieving an accuracy range of 0.86 to 0.887. Although regression tasks for estimating specific time-to-threshold values exhibit larger sensitivity to environment-specific data distributions, results confirm that initial trajectories of reward, entropy and policy loss hold significant predictive potential.

Our study also notes a significant data skew toward the 3DBall environment, which provided the most reliable training samples. Meanwhile, the metrics of smaller sample sizes in environments like Basic and WallJump may not fully reflect model generalization. Additionally, hardware variability is also a limiting factor in terms of time prediction, as regression models proved more sensitive than classification models.

This early-prediction system provides a valid pathway towards improving training efficiency. By identifying runs likely to fail within the initial 12% of their total duration, under-performing experiments can be terminated early, therefore conserving computational resources and reducing wasted time. Using the collected dataset, accounting for algorithmic, hyperparameter and hardware variability, the results provide strong evidence that the time-to-threshold is predictable through initial configuration and early learning signals, our model successfully distinguishing between failure and success trajectories long before the completion of training.

References

- C3.ai. (2025). *Classifier*. <https://c3.ai/glossary/data-science/classifier/>
- Domhan, T., Springenberg, J. T., & Hutter, F. (July, 2015). Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. *IJCAI (Vol. 15, pp. 3460-8)*.
- Geeks For Geeks. (13 Jan, 2025). *Regression in machine learning*. <https://www.geeksforgeeks.org/machine-learning/regression-in-machine-learning/>
- Unity Technologies. (2021). *ML-agents documentation*. <https://docs.unity3d.com/Packages/com.unity.ml-agents@3.0/manual/index.html>
- Unity Technologies. (2025). *Unity ml-agents toolkit [computer software]*. <https://github.com/Unity-Technologies/ml-agents>

A Column Dictionary

Column	Type	Description
<code>run_id</code>	string	Unique identifier for a single training run.
<code>environment</code>	string	Unity ML-Agents environment name (<code>3DBall</code> , <code>Basic</code> , <code>etc...</code>)
<code>seed</code>	int	Seed passed to the trainer used for run replication if needed.
<code>num_agents</code>	int	Number of parallel agents acting in environment.
<code>algorithm</code>	string	Learning algorithm used by trainer (<code>PPO</code> , <code>SAC</code>).
<code>steps</code>	int	Total number of training steps completed in this run.
<code>batch_size</code>	int	Batch size used per gradient update during training.
<code>buffer_size</code>	int	Size of the buffer (number of steps collected before each update).
<code>learning_rate</code>	float	Learning rate used when updating algorithm.
<code>epochs</code>	int	Number of passes over the buffer per update.
<code>total_time</code>	float	Wall clock duration of the run (seconds).
<code>average_cpu</code>	float	Mean CPU utilization over the course of the run.
<code>average_ram</code>	float	Mean RAM utilization over the course of the run.
<code>reward_mean</code>	float	Mean training reward logged by ML-Agents.
<code>reward_mean_step</code>	int	Training step index at which <code>reward_mean</code> was recorded.
<code>early_reward_mean</code>	float	Mean training reward over an early training window (first 12 percent of run).
<code>early_reward_mean_step</code>	int	Step index for the early reward window.
<code>final_reward_mean</code>	float	Mean reward over the final training window near the end of the run.
<code>final_reward_mean_step</code>	int	Step index associated with <code>final_reward_mean</code> .
<code>best_reward</code>	float	Highest observed reward during the run.
<code>best_reward_step</code>	int	Training step index at which <code>best_reward</code> was achieved.
<code>step_interval</code>	int	Logging interval in steps between consecutive metric aggregations.
<code>p_loss_mean</code>	float	Mean policy loss.
<code>p_loss_mean_step</code>	int	Step index associated with <code>p_loss_mean</code> .
<code>v_loss_mean</code>	float	Mean value loss.
<code>v_loss_mean_step</code>	int	Step index associated with <code>v_loss_mean</code> .
<code>entropy_mean</code>	float	Mean policy entropy.
<code>entropy_mean_step</code>	int	Step index associated with <code>entropy_mean</code> .
<code>threshold_value</code>	float	Performance threshold target used for this run.
<code>steps_to_threshold</code>	int	Training steps required until the threshold was first reached.
<code>time_to_threshold</code>	float	Wall clock time to reach the threshold, in seconds.
<code>threshold_version</code>	string	Configuration of the threshold for this run.
<code>run_reached_threshold</code>	boolean	Flag indicating whether the run reached the specified threshold.
<code>notes</code>	string	Free form notes about the run.

B Collected Data Table

For definitions of all column headers, see Appendix A.

Run ID	Env	Algo	Steps	Time	CPU	RAM	Avg R.	Early R.	Best R.
C_OE_3DBall_1	3DBall	ppo	500000	243	47.4	71.7	2.2754	1.1438	98.4877
C_AP_Basic_15	Basic	ppo	500000	2873	51.6	81.3	0.7235	0.0485	0.9300
C_LN_WallJump_112	BigWallJump	ppo	2000000	2599	51.4	69.9	-1.1584	-1.1666	-0.7376
C_OE_Crawler_78	Crawler	ppo	1000000	804	36.8	74.2	-0.9912	-0.9966	44.6265
C_AP_Hallway_83	Hallway	ppo	10000000	12799	64.8	88.7	-0.6806	-0.8104	0.6622
C_AK_PushBlock_24	PushBlock	ppo	2000000	2316	53.0	84.0	2.1814	2.1814	4.9728
C_LF_3DBall_103	3DBall	ppo	500000	861	18.6	54.1	2.6602	1.2223	97.1914
U_AP_Basic_15_A	Basic	ppo	50000	288	53.8	86.9	0.4810	0.0325	0.9278
C_AM_WallJump_55	BigWallJump	ppo	20000000	1532	45.2	89.9	-1.1388	-1.1518	-1.0267
C_LF_Crawler_39	Crawler	ppo	10000000	9687	23.7	49.6	-0.9956	-0.9980	2040.1321
C_LN_Hallway_45	Hallway	ppo	10000000	12129	46.6	71.8	-0.7027	-0.9880	0.6691
C_AK_PushBlock_21	PushBlock	ppo	2000000	2713	54.7	83.2	3.0839	3.0839	4.9670
C_OE_3DBall_4	3DBall	ppo	500000	262	47.4	71.1	2.7260	1.1816	100.0000
U_AP_Basic_11_B	Basic	ppo	50000	380	61.8	86.6	0.0580	-0.0324	0.0700
U_AM_WallJump_56	BigWallJump	ppo	1000000	2619	33.9	85.7	-1.1880	-1.1166	-0.8497
C_OE_Crawler_76	Crawler	ppo	1000000	789	36.7	74.1	-0.9912	-0.9958	58.1640
C_AP_Hallway_82	Hallway	ppo	10000000	12561	64.7	89.1	-0.9854	-1.0000	0.6629
U_AK_PushBlock_23	PushBlock	ppo	2000000	2415	53.8	84.4	1.0014	1.0014	4.9425
U_OE_3DBall_9	3DBall	sac	500000	507	53.7	73.9	7.0196	0.7413	97.2343
C_AP_Basic_18	Basic	ppo	500000	2887	52.0	85.8	0.7644	0.1294	0.9300
C_AM_WallJump_58	BigWallJump	ppo	20000000	1498	45.6	89.7	-1.1204	-1.1460	-1.0283
U_LF_Crawler_35	Crawler	ppo	1000000	1029	22.6	64.5	-0.9727	-0.9968	44.2201
U_LN_Hallway_A_44	Hallway	ppo	10000000	20904	52.9	74.8	-0.9277	-0.9131	0.9401
C_AK_PushBlock_23	PushBlock	ppo	2000000	2543	53.4	84.6	1.6921	1.6921	4.9730
C_OE_3DBall_5	3DBall	ppo	500000	290	49.4	70.6	2.5931	1.1879	100.0000
C_AK_Basic_98	Basic	ppo	500000	1007	39.5	84.5	0.6425	0.0000	0.9300
C_AM_WallJump_52	BigWallJump	ppo	20000000	1173	49.6	89.8	-1.0617	-1.1475	-0.9826
U_LF_Crawler_32	Crawler	ppo	1000000	1033	22.8	64.3	-0.9441	-0.9962	45.1580
U_LN_Hallway_B_48	Hallway	ppo	10000000	10237	54.1	75.3	-0.6141	-0.7763	0.8735
C_AK_PushBlock_21	PushBlock	ppo	2000000	2713	54.7	83.2	3.0839	3.0839	4.9670
C_OE_3DBall_4	3DBall	ppo	500000	262	47.4	71.1	2.7260	1.1816	100.0000
C_LN_Basic_19	Basic	ppo	500000	3026	58.1	73.5	0.7321	0.1111	0.9300
C_AM_WallJump_54	BigWallJump	ppo	20000000	1522	47.7	91.1	-1.0828	-1.1259	-0.7121
C_LF_Crawler_38	Crawler	ppo	10000000	10811	26.3	72.3	-0.9953	-0.9981	2079.8659
U_LN_Hallway_A_41	Hallway	ppo	10000000	10505	55.2	79.1	-0.7405	-0.7975	0.8337
C_AM_PushBlock_127	PushBlock	ppo	2000000	3305	32.2	83.9	-0.2423	-0.2423	4.9533
C_OE_3DBall_8	3DBall	ppo	500000	284	46.1	72.2	3.0765	1.2375	98.5385
U_AP_Basic_12_B	Basic	ppo	50000	305	55.3	81.9	0.8316	0.1120	0.9300
U_AM_WallJump_53	BigWallJump	ppo	1000000	1480	33.5	88.3	-1.1392	-1.1792	-0.7348
U_LF_Crawler_32	Crawler	ppo	1000000	1033	22.8	64.3	-0.9441	-0.9962	45.1580

Table 7: Summary of selected runs with key metrics, where R is the reward metric

C Changes made to increase data diversity

- **Algorithm change** PPO is replaced by SAC.
- **Scaling of the learning rate** The learning rate is modified by factors of $\times 0.5$ and $\times 2$ in the Basic environment.
- **Capacity increase** Batch size and buffer size are increased in the Crawler environment.
- **Variation of the discount factor** The discount function v is changed from 0.99 to 0.95 in Crawler experiments.
- **Change in entropy coefficient** The entropy regularization coefficient is varied from 1×10^{-2} to 5×10^{-3} in Hallway runs.
- **Network size increase** The hidden layer size is increased from 128 to 256 units in WallJump experiments.